SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: *Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Design Critique Session

Code of Conduct:

I Shaik Mohammed Waseem Rayan (192373004)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action.

Signature of the student:
Shaik Mohammed Waseem Rayan

SIMATS ENGINEERING
ASSESSMENT TOOLS

1 A PDA is designed to recognize a^nb^n, but it fails for a^nb^nC^n. Critique the design and identify possible flaws in stack operations and state transitions. How would you improve it?

Critique of the PDA

A PDA designed for $L = \{a^n b^n \mid n \geq 1\}$cannot directly recognize $L = \{a^n b^n c^n \mid n \geq 1\}$ because a standard PDA ftas **only one stack**, and recognizing three equal quantities requires remembering the count of a, b, and c.

Possible Flaws

1. **Stack operation flaw**
   - The PDA pushes one stack symbol for every a.
   - It pops one symbol for every b.
   - After all b symbols are processed, the stack becomes empty.
   - Therefore, there is no information left to compare the number of c symbols with a.
2. **State transition flaw**
   - The PDA may have transitions only for a → push and b → pop.
   - It lacks appropriate transitions for processing c.
   - If a c is encountered after the stack is empty, the PDA rejects or gets stuck.
3. **Incorrect acceptance**
   - The PDA may accept as soon as all X symbols are popped after the bs.
   - It therefore accepts $a^n b^n$ without verifying that the number of cs is also n.

Example

For: aaabbbccc The stack operations are: Read a a a → Push X X X Read

b b b → Pop X X X

Now:Stack = empty

Input remaining = c c c

The PDA has lost the count of as, so it cannot verify: number of a = number of b = number of c

How to Improve It

A **single standard PDA cannot recognize**: $\{a^n b^n c^n \mid n \geq 1\}$ because tftis language is **not context-free**. Therefore, simply adding more stack transitions will not solve the problem.

The appropriate improvement is to use a **Turing Machine**, which can mark symbols and repeatedly compare them:

a → X

b → Y

c → Z

For each cycle:

1. Mark one a as X
2. Find and mark one b as Y
3. Find and mark one c as Z
4. Return to the beginning
5. Repeat
6. Accept only when all a, b, c are matched

Final Answer

The PDA fails because its stack information is exhausted after matching a^n with b^n. It cannot remember the number of a's while processing c's. Missing c-transitions and premature acceptance are the major flaws. Adding ordinary PDA transitions is not sufficient because

$\{a^n b^n c^n \mid n \geq 1\}$ is not a context-free language.

The design should therefore be replaced by a Turing Machine, which can repeatedly mark and compare a's, b's, and c's.

2 A student claims that Context-Free Grammars (CFGs)

E → E + T | T
T → T * F | F
F → (E) | id
can be directly converted into a Deterministic PDA (DPDA).Critically evaluate this statement with justification and suggest corrections if needed

Critical Evaluation
The student's statement is **not completely correct**.
The CFG
E → E + T | T
T → T * F | F
F → (E) | id
describes arithmetic expressions and is **ambiguous as written?** More precisely, it is **left-recursive**, but it is actually **unambiguous** for the usual operator-precedence interpretation. However, it **cannot be directly converted into a DPDA using the standard CFG-to-PDA construction**.
Why?
The standard CFG → PDA conversion produces a **non-deterministic PDA (NPDA)**.
For example, when the PDA has E on top of the stack, it must choose between:
E → E + T
E → T
Similarly:
T → T * F
T → F
The PDA must make these production choices using **ε-transitions**, without necessarily knowing which production will lead to a successful parse.
Therefore, the direct construction is generally:
CFG → NPDA
not automatically:
CFG → DPDA
Important Point

The language generated by this grammar **is deterministic context-free** under the standard expression syntax because its structure can be parsed using operator precedence.
However, a **DPDA requires a deterministic parsing strategy**, so the grammar should first be transformed into a suitable form.

Correction

Remove left recursion:

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → (E) | id

This grammar is better suited for predictive/deterministic parsing.

Final Evaluation

Claim	**Evaluation**
CFG can be converted to a PDA	✅ Yes
Direct standard conversion gives a DPDA	❌ No
Standard conversion gives an NPDA	✅ Yes
This expression language can be handled deterministically	✅ Yes
Grammar should be transformed for deterministic parsing	✅ Recommended

Conclusion

The student's claim is **incorrect as stated**. A CFG can be directly converted into an **NPDA**, but not necessarily into a **DPDA**. To obtain deterministic behavior, the grammar should be transformed to eliminate left recursion and then implemented using a deterministic parsing strategy such as **LL(1)** or an appropriate **LR-style** parser.

Exam-ready conclusion:

The direct CFG-to-PDA construction is nondeterministic because production rules are selected through ε-transitions. Hence, the given CFG cannot be directly converted to a DPDA by the standard method. For deterministic parsing, the grammar should first be transformed to remove left recursion and then a suitable deterministic parsing method should be applied.

3 A Turing Machine is designed using multiple tracks to solve a problem. Suggest

alternative programming techniques like checking-off symbols or subroutines to do subtraction

A multi-track Turing Machine can perform subtraction, but the same problem can be solved using simpler **single-tape programming techniques**.

Assume the input is in unary form:

11111#111 = 5 - 3

The expected output is:

11 = 2

1. Checking-Off Symbols

Use special markers to indicate symbols that have already been processed.

Input: 11111#111

↓

Mark pairs: X X X # Y Y Y

↓

Remaining: 11

Method:

1. Find an unmarked 1 before # and mark it X.
2. Move right and find an unmarked 1 after #; mark it Y.
3. Return to the left and repeat.
4. When all 1s after # are marked, the remaining unmarked 1s represent the difference.

5. Erase unnecessary markers and halt.

Advantage: Simple and easy to trace.

2. Subroutine Technique

Divide the subtraction process into reusable subroutines.

SUBTRACT:

Find an unmarked 1 in first number

Find an unmarked 1 in second number

Mark both

Repeat

Output remaining 1s

Possible subroutines:

FIND-A → Find an unmarked 1 in the first number

FIND-B → Find an unmarked 1 in the second number

MARK → Mark the selected symbols

RETURN → Return to the beginning

CLEAN → Remove markers

Advantage: Makes the TM easier to design, debug, and understand.

3. Repeated Cancellation

Another technique is to **cancel one symbol from each number repeatedly**.

11111#111

↓

X1111#Y11

↓

XX111#YY1

↓

XXX11#YYY

The unmatched symbols on the left are:

11

Therefore:

5 - 3 = 2

Comparison

Technique	**Main Idea**	**Advantage**
Checking-off	Mark processed symbols	Simple
Subroutines	Divide TM into reusable operations	Modular
Cancellation	Remove one symbol from each number	Easy to visualize

Technique	Main Idea	Advantage
Multiple tracks	Store related information separately	More organized but complex

Conclusion

A multi-track TM is **not necessary** for subtraction. **Checking-off/cancellation** is the simplest alternative, while **subroutines** provide a cleaner and more modular TM design. Both reduce the complexity of managing multiple tracks.

4 While designing a PDA accepts the language

$$L = \{w \mid w \in (a + b)^* \text{ and } n_a(w) == n_b(w)\}$$

$n_a(w)$ is the number of a's in w

$n_b(w)$ is the number of b's in w, acceptance is defined only by final state, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

The language is:
That means the PDA must accept strings having an **equal number of a and b**, regardless of their order.

Critique

Accepting **only by final state** and ignoring empty-stack acceptance **does not change the language**, provided the PDA is correctly designed.
For example:
ab → equal a's and b's → Accept
ba → equal a's and b's → Accept
aabb → equal → Accept
abab → equal → Accept
A PDA can be designed to reach a final state exactly when all unmatched symbols have been cancelled.

Important Point

For **general PDAs**, acceptance by:

1. Final state
2. Empty stack

are equivalent in terms of the class of languages that can be recognized (context-free languages), although the **particular PDA's behavior** can differ.

So, if the PDA is correctly converted to use final-state acceptance, **the recognized language need not change**.

Possible Design Problem

The real issue is whether the stack operations correctly handle both cases:

a followed by b

b followed by a

For example:

Read a → push A

Read b → if A is on top, pop A

Read b → push B

Read a → if B is on top, pop B

This allows the PDA to cancel unmatched as and bs regardless of their order.

Conclusion

Final-state acceptance alone does NOT inherently change the language.

If the PDA is correctly constructed, it can still recognize:

$L = \{ w \in \{a,b\}^* \mid n_a(w) = n_b(w) \}$

However, the stack transitions must correctly cancel unmatched a's and b's. If the PDA relies only on an empty-stack condition without properly converting the acceptance mechanism, its particular behavior may change.

Exam-ready answer:

The design choice of accepting only by final state is not a fundamental problem. Final-state and empty-stack acceptance have equivalent expressive power for general PDAs. Therefore, the language need not change. The important requirement is that the stack transitions correctly match/cancel a and b in any order so that the PDA reaches the final state exactly when .

5 A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare FA, PDA, and Turing Machine capabilities and evaluate whether this replacement is valid.

A system designer suggests replacing a **Turing Machine (TM)** with a **Pushdown Automaton (PDA)** to reduce computational complexity.

FA:

Can recognize:

$(a+b)^*$

but cannot recognize:

$\{a^n b^n \mid n \geq 1\}$

because it has no memory to count the as.

PDA:

Can recognize:

$\{a^n b^n \mid n \geq 1\}$

because it can push symbols for as and pop them for bs.

But it cannot recognize:

$\{a^n b^n c^n \mid n \geq 1\}$

because one stack is insufficient for comparing three independent counts.

Turing Machine:

Can recognize both:

$\{a^n b^n\}$

$\{a^n b^n c^n\}$

and can perform general algorithms and computations.

Is the Replacement Valid?

Not always.

Replacing a TM with a PDA is valid **only when the problem belongs to the class of context-free languages and does not require the additional computational power of a TM**.

For example:

a^nb^n → PDA is sufficient

But:

$a^nb^nc^n$ → PDA is NOT sufficient

General computation → PDA is NOT sufficient

Final Conclusion

A PDA can reduce computational resources for problems that require only stack-based memory, but it cannot replace a Turing Machine for general computation. Therefore, the proposed replacement is valid only for suitable context-free problems; otherwise, it results in loss of computational capability.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand